

Docker Practical Assignment

Prepared by: Vimala Ankumpeta

Topics: Docker, Architecture, Images, Containers, Dockerfile, Volumes, Networks, Compose and Docker Hub

1. What is Docker?

Docker is an open-source containerization platform used to package an application together with its libraries, dependencies and configuration into a lightweight, portable container. Containers provide a consistent environment across development, testing and deployment, reducing the common problem of applications behaving differently on different systems.

Key benefits: portability, isolation, fast startup, efficient resource usage and repeatable deployments.

Docker Desktop Installation: <https://docs.docker.com/desktop/setup/install/windows-install/>

2. Docker Architecture

Docker follows a client-server architecture. The Docker CLI acts as the client and communicates with the Docker Engine. The Docker daemon manages images, containers, networks and volumes. Docker Hub or another registry can be used to store and distribute images.

Component	Purpose
Docker Client / CLI	Accepts commands such as docker build, docker run and docker pull.
Docker Engine / Daemon	Creates and manages containers, images, networks and volumes.
Docker Images	Read-only templates used to create containers.
Docker Containers	Running, isolated instances created from images.
Docker Registry	Stores and distributes Docker images; Docker Hub is a common public registry.

3. Images vs Containers

A Docker image is a packaged, read-only template containing the application and everything needed to run it. A container is a running or stopped instance created from an image. Multiple containers can be created from the same image.

Docker Image	Docker Container
Template for an application	Instance created from an image
Generally read-only	Has a writable container layer
Used to create containers	Used to run the application
Can be stored in Docker Hub	Runs on a Docker host

4. Dockerfile

A Dockerfile is a text file containing instructions used to build a Docker image. It defines the base image, application files, dependencies, ports and other configuration needed to create the image.

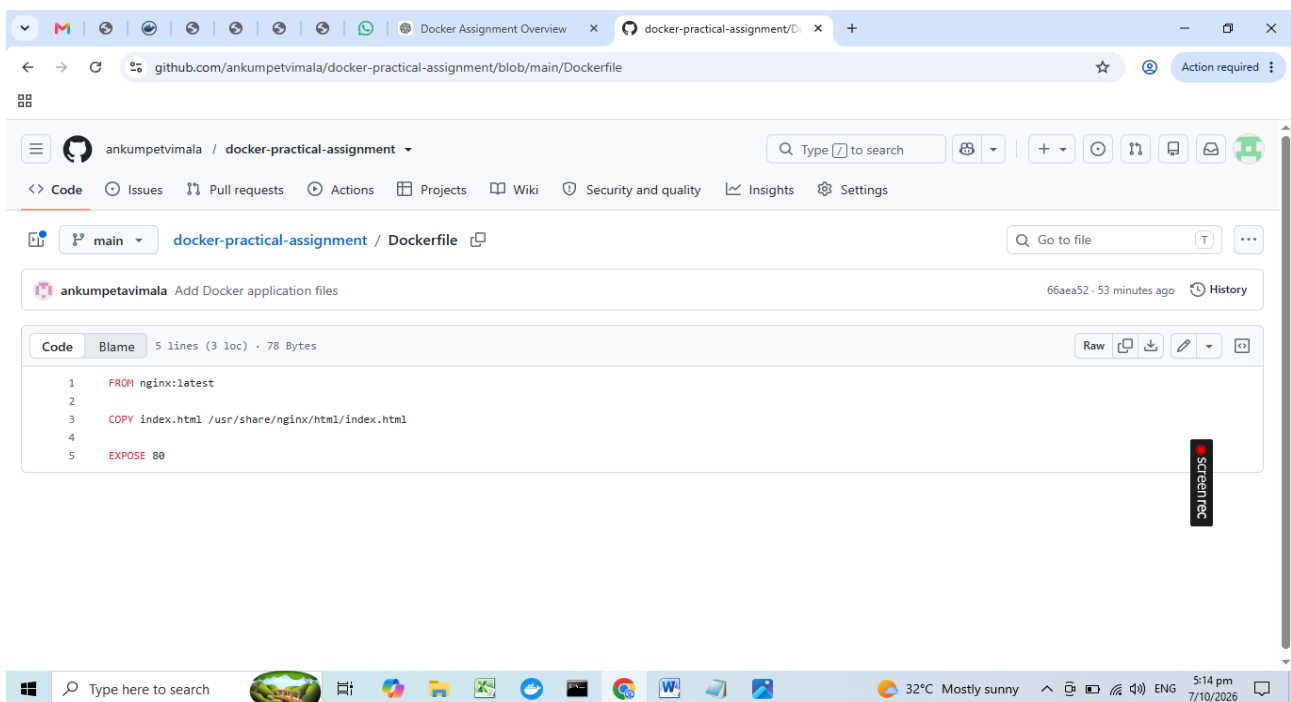
Example:

```
FROM nginx:latest
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
EXPOSE 80
```

The image can then be built using **docker build -t my-web-app:1.0 ..**



5. Docker Volumes

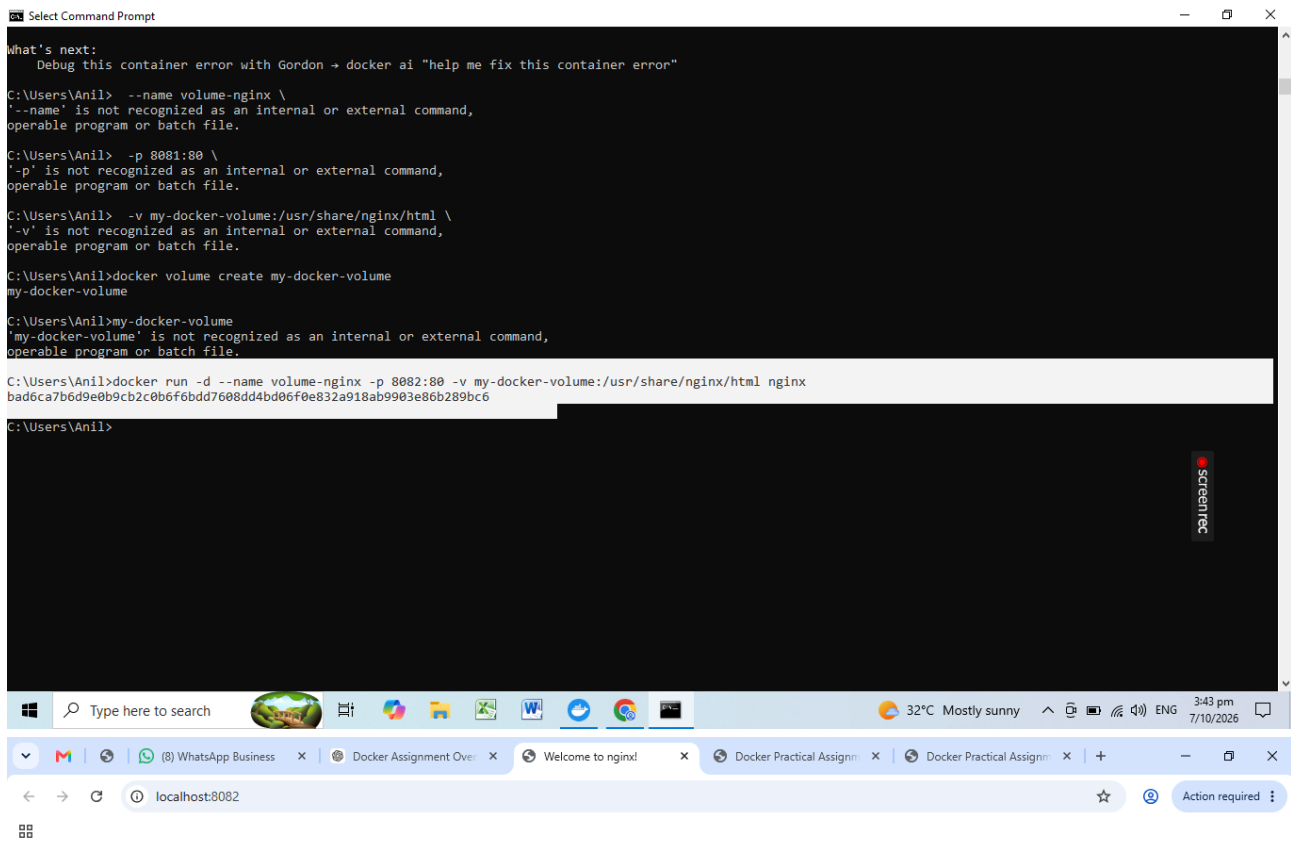
Docker volumes provide persistent storage for container data. Container storage is normally tied to the container lifecycle, whereas a volume exists independently and can be attached to one or more containers as required.

Example:

```
docker volume create my-volume
```

```
docker run -d --name web -v my-volume:/usr/share/nginx/html nginx
```

Benefits: persistent data, easier backups, separation of application data from containers, and convenient data sharing.



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

6. Docker Networks

Docker networks allow containers to communicate with each other and control connectivity between containers and external networks. A user-defined bridge network is commonly used for applications where multiple containers need to communicate.

Example:

```
docker network create my-network
docker run -d --name web --network my-network nginx
```

Containers connected to the same user-defined network can communicate using container names as DNS names.

```
Select Command Prompt
'be15093df5d' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Anil>docker run -d --name web1 --network my-docker-network nginx
f4d6d789c918959ab61c9793b7d8d1e2902badea177f4ec7444ccc4ce036a270

C:\Users\Anil>docker run -d --name web2 --network my-docker-network nginx
c5e89be17cfe2e0683b17e4864e02e08a79751acba668e502b3d5568bae43063

C:\Users\Anil>docker network inspect my-docker-network
{
  "Name": "my-docker-network",
  "Id": "be15093df5d08b77442f6133e350643ab7b7d51ff97f2b16c3b24effb303b7a",
  "Created": "2026-10-07T10:17:03.201637698Z",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv4": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": {},
    "Config": [
      {
        "Subnet": "172.18.0.0/16",
        "Gateway": "172.18.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Options": {
    "com.docker.network.enable_ipv4": "true",
    "com.docker.network.enable_ipv6": "false"
  },
  "Labels": {},
  "Containers": {
    "c5e89be17cfe2e0683b17e4864e02e08a79751acba668e502b3d5568bae43063": {
      "Name": "web2",

```

7. Docker Compose

Docker Compose is a tool for defining and running multi-container applications using a YAML configuration file. A Compose file can define services, images, builds, ports, volumes and networks in one place.

Example:

```
services:
  web:
    build: .
    ports:
      - "8080:80"
  redis:
    image: redis:latest
```

The application can be started with **docker compose up -d --build** and stopped with **docker compose down**.

```
Command Prompt
What's next:
  Debug this Compose error with Gordon → docker ai "help me fix this compose error"

C:\Users\Anil>notepad docker-compose.yml

C:\Users\Anil>docker compose up -d --build
[+] Building 2.6s (9/9) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 465B 0.0s
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 119B 0.1s
=> [internal] load metadata for docker.io/library/nginx:latest 0.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 32B 0.0s
=> [1/2] FROM docker.io/library/nginx:latest@sha256:f9ea18bfa4fad859e1ed38259d711da7ccad2c3516e875cec3351a57c859f571 0.1s
=> => resolve docker.io/library/nginx:latest@sha256:f9ea18bfa4fad859e1ed38259d711da7ccad2c3516e875cec3351a57c859f571 0.1s
=> CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> exporting to image 0.4s
=> => exporting layers 0.0s
=> => exporting manifest sha256:8cbdf1bf45ef14141a091f3cb44624ba9fbfc97e8e13257091733ba4f5999b9c 0.0s
=> => exporting config sha256:888924abd6308f3b43e3bc3e589b18e198a28984875b82276e5c4d1575aa1b51 0.0s
=> => exporting attestation manifest sha256:cbc7c8f8a669c1d5b4f0d70ffce571291932b349e9477ea70d9d23e5349636c5 0.1s
=> => exporting manifest list sha256:7a0ffbf3040a3ba4d9c208adb1363673818bec318e15fba6a8f68412dc23dfa 0.0s
=> => naming to docker.io/library/anil-web:latest 0.0s
=> => unpacking to docker.io/library/anil-web:latest 0.0s
=> resolving provenance for metadata file 0.1s
[+] up 3/3
Image anil-web Built 3.1s
Container docker-redis Running 0.0s
Container docker-web Started 1.3s

C:\Users\Anil>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
lac88aab825a   anil-web   "/docker-entrypoint..." 39 seconds ago Up 38 seconds 0.0.0.0:8083->80/tcp, [::]:8083->80/tcp  docker-web
caed653de6a8   redis:late "docker-entrypoint.s..." 4 minutes ago Up 4 minutes   6379/tcp                                docker-redis
c5e89be17cfe   nginx     "/docker-entrypoint..." 8 minutes ago Up 8 minutes   80/tcp                                  web2
f4d6d789c918   nginx     "/docker-entrypoint..." 8 minutes ago Up 8 minutes   80/tcp                                  web1
ba66ca7b6d9e   nginx     "/docker-entrypoint..." 12 minutes ago Up 12 minutes  0.0.0.0:8082->80/tcp, [::]:8082->80/tcp  volume-nginx
e3e6d899f7b6   nginx     "/docker-entrypoint..." 38 minutes ago Up 38 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  my-nginx
```

8. Docker Hub

Docker Hub is a cloud-based container registry used to find, store, share and distribute Docker images. Developers can pull public images such as Nginx or Redis and can also push their own images to a Docker Hub repository.

Typical workflow:

```
docker login
docker tag my-web-app:1.0 username/my-web-app:1.0
docker push username/my-web-app:1.0
```

Docker hub Image

The screenshot is divided into two main sections: a terminal window at the top and the Docker Hub web interface at the bottom.

Terminal Window:

```
Select Command Prompt
C:\Users\Anil>docker compose down
[+] down 3/3
  Container docker-web      Removed
  Container docker-redis    Removed
  Network anil_app-network  Removed

C:\Users\Anil>docker compose down -v
[+] down 1/1
  Volume anil_web-data      Removed

C:\Users\Anil>docker login
Authenticating with existing credentials... [Username: ankumpetavimala]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded

C:\Users\Anil>docker tag vimala-docker-app:1.0 ankumpetavimala/vimala-docker-app:1.0

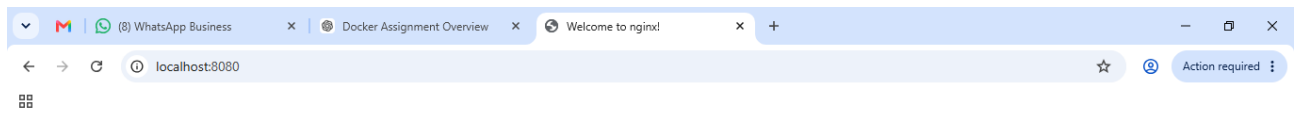
C:\Users\Anil>docker push ankumpetavimala/vimala-docker-app:1.0
The push refers to repository [docker.io/ankumpetavimala/vimala-docker-app]
04bdbc7ea6b5: Mounted from library/nginx
d20f1f13f4db: Pushed
44136fa355b3: Pushed
ecc510c1e359: Mounted from library/nginx
807ab930f3a0: Mounted from library/nginx
1efc53192927: Mounted from library/nginx
947d22046304: Mounted from library/nginx
8020170dc65f: Mounted from library/nginx
f7e1db57b58b: Mounted from library/nginx
273471a3557f: Pushed
1.0: digest: sha256:32f05e751f2ce3617c66873cbb41635b61ff46aae41af9e310edb0fce764f18 size: 856

C:\Users\Anil>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
anil-web:latest	7a0ffbf3040a	240MB	63.9MB	
ankumpetavimala/vimala-docker-app:1.0	32f05e751f2c	240MB	63.9MB	
nginx:latest	f9ea18bfa4fa	243MB	66.7MB	U
redis:latest	c94085d298b7	213MB	57.6MB	
vimala-docker-app:1.0	32f05e751f2c	240MB	63.9MB	

Docker Hub Web Interface:

The browser shows the Docker Hub page for the repository `ankumpetavimala/vimala-docker-app`. The page includes the repository name, a description "By ankumpetavimala · Updated 2 minutes ago", and a "Manage Repository" button. Below the repository information, there is a section titled "How likely are you to recommend Docker Hub to another developer?" with a rating scale from 0 to 10. The page also features a search bar and navigation links like "Explore" and "My Hub".



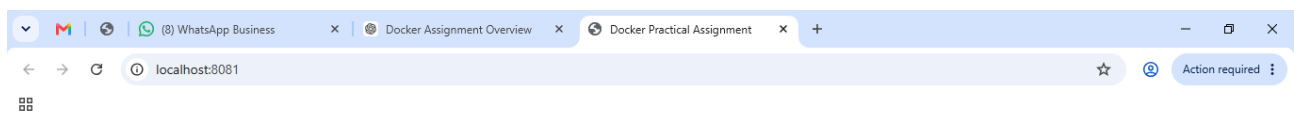
Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Screenred



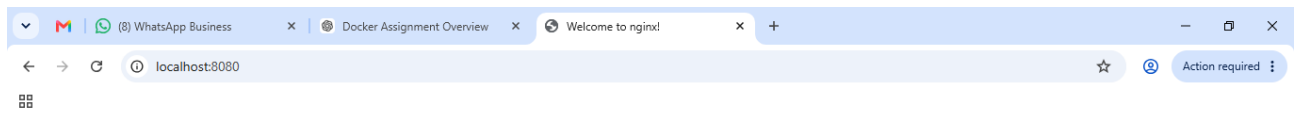
Welcome to My Docker Application

This application is running inside a Docker container.

Created by Vimala Ankumpeta

Screenred





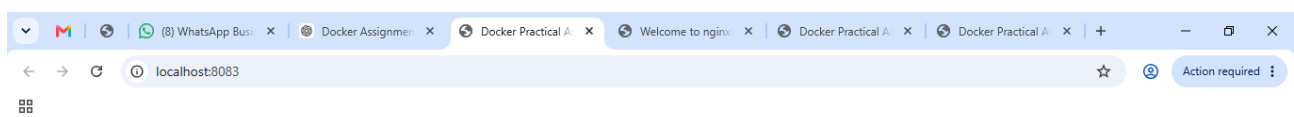
Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Screenred



Welcome to My Docker Application

This application is running inside a Docker container.

Created by Vimala Ankumpeta

Screenred



Conclusion

Docker provides a practical way to package and run applications consistently. Images define what an application needs, containers run those images, volumes provide persistent data, networks enable container communication, and Docker Compose simplifies multi-container deployments. Docker Hub provides a convenient registry for sharing and distributing images.